AMCAT Coding Questions (CSE / Automata)

Compiled from public third-party prep sources (PrepInsta, Scribd uploads). AMCAT publishes no official past papers, so questions are paraphrased, are not guaranteed to be from a specific year, and are unofficial. Solutions are Python, written and run against the sample cases listed. Format notes (FACE Prep): Automata = 2 coding questions (1 easy, 1 hard), ~30 min; Automata Fix = 7 questions, 20 min, no Python. Not covered here: cellular automaton, sorted circular linked list insert, LRU cache misses, round-robin waiting time, maze path (named in a Scribd list, statements not retrieved).

Q1. Fountain Installation

Source: PrepInsta

1-D garden of length n, one fountain at each position; fountain i covers radius locations[i], clipped to the garden. Find the minimum fountains to activate to cover the whole garden. 1<=n<=1e5, 0<=locations[i]<=min(n,100).

Sample: n=3, locations={0,2,1} -> 1

```
def fountain(n, loc):
    iv = [(max(1, i+1-r), min(n, i+1+r)) for i, r in enumerate(loc)]
    best = [0]*(n+2)
    for l, r in iv: best[l] = max(best[l], r)
    cnt = cur = far = 0
    for p in range(1, n+1):
        far = max(far, best[p])
        if p > cur:
            if far < p: return -1
            cnt += 1; cur = far
    return cnt
```

Q2. Collecting Candies

Source: PrepInsta

N boxes of candies. Merging two boxes costs the sum of their candies. Minimum total time to merge all into one box. T test cases; 1<N<10000.

Sample: {1,2,3,4} -> 19. Source also lists {1,2,3,4,5} -> 34, but the optimal (Huffman) merge gives 33 - treat the source value as a likely error.

```
def candies(a):
    heapq.heapify(a); t = 0
    while len(a) > 1:
        s = heapq.heappop(a) + heapq.heappop(a); t += s; heapq.heappush(a, s)
    return t
```

Q3. Class Monitor

Source: PrepInsta

A student's rank is recorded at each visit. A name is cut whenever a rank is lower than every earlier rank. Count cuts (the first rank is never a cut).

Sample: 4 3 7 2 6 1 -> 3

```
def monitor(ranks):
    cut, lo = 0, ranks[0]
    for r in ranks[1:]:
        if r < lo: lo = r; cut += 1
    return cut
```

Q4. Seating Arrangement in Exam Hall

Source: PrepInsta

List binary strings (length>=1, starting with 1) with no two adjacent 1s, ordered by length then lexicographically: 1, 10, 100, 101, 1000, 1001, 1010, ... For each query k print the k-th string.

Sample: k=4,6,9 -> 101, 1001, 10001

```
def kth_seat(k):
    q = deque(["1"]); out = []
    while len(out) < k:
        s = q.popleft(); out.append(s)
        q.append(s+"0")
        if s[-1] == "0": q.append(s+"1")
    return out[k-1]
```

Q5. Formatting Large Products

Source: PrepInsta

Given 1<=A<=B<=1e6, write the product A*(A+1)*...*B as D * 10^E with D's last digit non-zero. (Reference solution uses big ints; at B=1e6 count factors of 2 and 5 instead.)

Sample: (1,5) -> 12 * 10^1; (3,10) -> 18144 * 10^2

```
def big_product(a, b):
    p = 1
    for i in range(a, b+1): p *= i
    e = 0
    while p % 10 == 0: p //= 10; e += 1
    return f"{p} * 10^{e}"
```

Q6. Left Rotation

Source: PrepInsta

Perform d left rotations on an array of n integers and print it. 1<=n<=1e5, 1<=d<=n.

Sample: {1,2,3,4,5}, d=4 -> 5 1 2 3 4

```
def rotate_left(arr, d):
    d %= len(arr); return arr[d:] + arr[:d]
```

Q7. Vowel Encryption

Source: PrepInsta (rule inferred from one sample)

Map each distinct digit of a number to a letter: prime digits (2,3,5,7) get vowels, other digits get consonants, assigned in order of first appearance so the result is lexicographically smallest. Print the encoded string.

Sample: 123421 -> baecab

```
def vowel_encrypt(num):
    vowels, cons = iter("aeiou"), iter("bcdfghjklmnpqrstvwxyz")
    m = {}
    for ch in num:
        if ch not in m: m[ch] = next(vowels) if ch in "2357" else next(cons)
    return "".join(m[c] for c in num)
```

Q8. Array Sub Array

Source: PrepInsta

Given window length x and array of n integers, take the minimum of every contiguous window of length x, then output the maximum of those minimums.

Sample: x=1, {1,2,3,1,2} -> 3

```
def max_of_window_mins(x, a):
    dq, mins = deque(), []
    for i, v in enumerate(a):
        while dq and a[dq[-1]] >= v: dq.pop()
        dq.append(i)
        if dq[0] <= i-x: dq.popleft()
        if i >= x-1: mins.append(a[dq[0]])
    return max(mins)
```

Q9. Queries for Count

Source: PrepInsta

Unsorted array; for each query [i,j] count elements with i<=value<=j.

Sample: [1,3,3,9,10,4]: [1,4] -> 4, [9,12] -> 2

```
def range_counts(arr, queries):
    from bisect import bisect_left, bisect_right
    s = sorted(arr)
    return [bisect_right(s, j)-bisect_left(s, i) for i, j in queries]
```

Q10. Print Matrix in Spiral Form

Source: Scribd compilation

Print a 2D array in spiral order.

Sample: [[1,2,3],[4,5,6],[7,8,9]] -> 1 2 3 6 9 8 7 4 5

```
def spiral(m):
    out = []
    while m:
        out += m.pop(0)
        m = [list(r) for r in zip(*m)][::-1]
    return out
```

Q11. Row with Most Zeroes

Source: Scribd compilation

Find the row of a 0/1 matrix with the most zeroes.

Sample: [[1,0,0],[0,0,0],[1,1,0]] -> row 1

```
def row_most_zeros(m):
    return max(range(len(m)), key=lambda i: m[i].count(0))
```

Q12. Pythagorean Triplets up to a Limit

Source: Scribd compilation

Print all Pythagorean triplets (a,b,c) with c<=limit.

Sample: limit=20 -> (3,4,5),(6,8,10),(5,12,13),(9,12,15),(8,15,17),(12,16,20)

```
def pythag(limit):
    return [(a,b,c) for c in range(1,limit+1) for b in range(1,c) for a in range(1,b) if a*a+b*b==c*c]
```

Q13. Maximum Contiguous Subarray Sum

Source: Scribd compilation

Find the maximum sum of a contiguous subarray.

Sample: {-2,1,-3,4,-1,2,1,-5,4} -> 6

```
def kadane(a):
    best = cur = a[0]
    for v in a[1:]:
        cur = max(v, cur+v); best = max(best, cur)
    return best
```